Names:ISHIMWE GHISLAIN
ID:20251SEN170

ADVENTIST UNIERSITY CENTRAL OF AFRICA

Database Management System Assignment

1.

a**)**

- Repetition of patient names: The name *John Doe* appears in multiple appointment records, which may lead to data inconsistency if changes are not applied everywhere.
- Repeated doctor names: The name *Dr. Smith* is stored in several rows (e.g., P101, P102). Updating doctor details may require modifying multiple records.
- Duplicate specialization entries: The specialization *Cardiology* is repeated for the same doctor across different rows, increasing the risk of inconsistent updates.
-

b) Insertion Anomaly

It is not possible to insert a new doctor into the system unless they are associated with a patient, since both **Doctor_ID** and **Patient_ID** form a composite primary key. This limits adding independent doctor records.

c) Deletion Anomaly

Deleting a patient record could result in losing important doctor-related data such as **Doctor_ID** and **specialization**, even though the doctor may still exist in the system.

d) Update Anomaly

Updating a doctor's details requires modifying multiple rows. If one record is missed, it can lead to inconsistent or incorrect data.

2.

a) Functional Dependencies

i. Salesperson_ID → Salesperson_Name
ii. Product_ID → Product_Name
iii. Customer_ID → Customer_Name
iv. (Salesperson_ID, Product_ID, Customer_ID, Sales_Date) → Quantity

b) Primary Key

The composite primary key is:
(Salesperson_ID, Product_ID, Customer_ID, Sales_Date)

c) Partial Dependencies

- Salesperson_Name depends only on Salesperson_ID
- Product_Name depends only on Product_ID
- Customer_Name depends only on Customer_ID

d) Transitive Dependencies

There are **no transitive dependencies** in this relation.

3.

a) Functional Dependencies

- OrderID → CustomerName, CustomerPhone
- ProductID → ProductName, ProductPrice
- (OrderID, ProductID) → Quantity
-

b) Additional Dependencies

- OrderID → CustomerName, CustomerPhone, OrderDate
- ProductID → ProductName, ProductPrice

c)

Orders

OrderId	CustomerName	CustomerPhone	OrderDate

Products

ProductId	ProductName	ProductPrice

OrderItems

OrderId	ProductId	Quantity

d)

Table	Primary Key	Foreign Key
Orders	OrderId	____
Products	ProductId	_____
OrderItems	OrderId,ProductId	OrderId-->Orders ProductId-->Product

e) Sample data for new tables:

Orders

OrderId	CustomerName	CustomerPhone	OrderDate
O100	John Doe	0788123456	2025-01-10
O101	Jane Roe	0788234567	2025-01-11

Products

ProductId	ProductName	ProductPrice
P10	Laptop	500000
P20	Mouse	15000

OrderItems

OrderId	ProductId	Quantity
O100	P10	2
O100	P20	3
O101	P10	1

3) a) -EmployeeID → EmployeeName, DepartmentID, Salary
-DepartmentID → DepartmentName, DepartmentLocation, ManagerName
-EmployeeID → DepartmentName, DepartmentLocation, ManagerName

b) Transitive Dependencies:
-EmployeeID → DepartmentID → DepartmentName
-EmployeeID → DepartmentID → DepartmentLocation
-EmployeeID → DepartmentID → ManagerName

Why it violates Third Normal Form (3NF):

- **Redundancy of department data:** Department details are stored repeatedly for every employee belonging to the same department.
- **Update anomaly:** Modifying department information (such as location) requires updating multiple records, which can lead to inconsistencies.
- **Insertion anomaly:** A new department cannot be added unless there is at least one employee assigned to it.
- **Deletion anomaly:** Removing the last employee of a department may result in the loss of that department's information.

c) 3NF Decomposition:

EMPLOYEE

EmployeeId	EmployeeName	DepartmentId	Salary
E100	Alice	D10	800000
E101	Bob	D10	750000
E102	Carol	D20	700000
E103	David	D20	720000

DEPARTMENT

DepartmentId	DepartmentName	DepartmentLocation	ManagerName
D10	IT	Building A	Bob Smith
D20	HR	Building B	Jane Doe

d) Primary and Foreign keys:

EMPLOYEE
-Primary Key: EmployeeId
-Foreign Key: DepartmentId

DEPARTMENT
-Primary key: DepartmentId

e) Elimination of Anomalies

- Update Anomaly:
  Before: Updating the IT department location required modifying multiple records.
  After: The change is made in a single row within the **Department** table.

- Insertion Anomaly:
  Before: It was not possible to add a new department without assigning employees to it.
  After: New departments can be added independently in the **Department** table.

- Deletion Anomaly:
  Before: Deleting the last employee in the HR department could result in losing all HR-related information.
  After: The **Department** table preserves department details even if no employees exist.

f)

EMPLOYEE Table

EmployeeId	EmployeeName	DepartmentId	Salary
E100	Alice	D10	800000
E101	Bob	D10	750000
E102	Carol	D20	700000
E103	David	D20	720000

DEPARTMENT Table

DepartmentId	DepartmentName	DepartmentLocation	ManagerName
D10	IT	Building A	Bob Smith
D20	HR	Building B	Jane Doe

4) a) Students majoring in computer science:

π Name, GPA (σ GPA > 3.7 (STUDENT))

b) Course names that are worth 4 credits:

π Name, GPA (σ GPA > 3.7 (STUDENT))

c) Names and GPA's of all students with GPA above 3.7:

π Name, GPA (σ GPA > 3.7 (STUDENT))

d) Courses in computer Science with more than 3 credits:

π CourseID, CourseName, Credits (σ Department = 'CS' $\wedge$ Credits > 3 (COURSE))

e) Students who enrolled in any course during fall 2024:

STUDENT $\ltimes$ (σ Semester = 'Fall2024' (ENROLLMENT))

6) a) Names of students enrolled in 'CS101':

π Name (STUDENT ⋈ (σ CourseID = 'CS101' (ENROLLMENT)))

b) Names of students who got an 'A' grade:

π Name (STUDENT ⋈ (σ Grade = 'A' (ENROLLMENT)))

c) CS courses that students are enrolled in:

π CourseName (COURSE ⋈ (σ Department = 'CS' (COURSE) ⋈ ENROLLMENT))

d) Names of students taking courses worth 4 credits:

-Step 1: Select courses with 4 Credits

T1:σ credits=4(Course)

-Step 2: Join with Enrolled to find those who takes those courses:

T2=T1⋈Enrolled

-Step 3:Join with students to get names

T3=T2⋈Student

Step 4: π name((σCredits=4(Course)⋈Enrolled)⋈Students)

e) Find the names of all Computer Science majors who are enrolled in the Database course

Step 1: Select CS majors

T1=σmajor='CS'(students)

Step 2: Find Database course

T2=σ title='Database'(Course)

Step 3: Join T2 with Enrolled to get enrollments in Database

T3=T2⋈Enrolled

Step 4: Join with CS majors

T4=T3⋈T1

Step 5: Project names

π name(T4)

Final Expression:

πname((σtitle='Database'(COURSE)⋈Enrolled)⋈σmajor'CS'(Student))

7) a) 1NF violation: The table contains multi valued attributes.

2NF violation: Some attributes depend only on part of the key.

3NF violation:Some attributes depend on other non-key attributes.

b)Step 1: Convert to 1NF

Remove repeating groups and ensure atomic values.

CUSTOMERS (Primary Key=CustomerId)

CustomerId	CustomerName	Customerphone	CustomerCity
C1	John	0788123456	Kigali
C2	Jane	0788234567	Butare

SELLERS (Primary Key=Seller_Id)

SellerId	SellerName	Selleremail
S1	Alice	alice@store.rw
S2	Bob	bob@store.rw

PRODUCTS(Primary Key=ProductId)

ProductId	ProductName	UnitPrice
P1	Laptop	500000
P2	Mouse	15000
P3	Phone	300000

INVOICES(PK=InvoiceId FK=CustomerId,SellerId

InvoiceId	Date	CustomerId	SellerId
I001	2025-01-10	C1	S1
I002	2025-01-11	C2	S2
I003	2025-01-12	C1	S1

PRODUCT_SALES(PK=InvoiceId,ProductId FK=InvoiceId,ProductId)

InvoiceId	ProductId	Quantity
I001	P1	1
I001	P2	2
I002	P3	1
I003	P3	2

c) Final 3NF Design
CUSTOMER
PK → CustomerPhone (CustomerPhone, CustomerName, CustomerCity)

SELLER
PK → SellerEmail (SellerEmail, SellerName)

PRODUCT
PK → ProductID (ProductID, ProductName)

INVOICE
PK → InvoiceID
FK → CustomerPhone
FK → SellerEmail
(InvoiceID, Date, CustomerPhone, SellerEmail)
PRODUCT_SALES
PK → (InvoiceID, ProductID)
FK → InvoiceID
FK → ProductID
(InvoiceID, ProductID, Quantity, UnitPrice)

d)Relational Algebra Queries

1) All customers from Kigali:

Σ customercity='KIGALI'(CUSTOMER)

2) All products sold by Alice

Step 1:

σsellerName='Alice'(SELLER)

Step 2:
Join with invoice
SELLER⋈INVOICE
Step 3:
Join with product sales and product
π productName((SELLER⋈INVOICE)⋈PRODUCT_SALES⋈PRODUCT)

3) Names of Customers who bought phones
Step 1:
σ product_name='phone'(PRODUCT)

Step 2:
Join Tables
πcustomerName(CUSTOMER⋈INVOICE⋈PRODUCT_SALES⋈PRODUCT)

e) How Normalization Prevents Anomalies

Insertion Anomaly

- *Before normalization:* It was not possible to add a new product unless an invoice record was also created.
- *After normalization:* Product details are stored separately in the **PRODUCT** table, allowing new products to be inserted independently.

Update Anomaly

- *Before normalization:* Updating a seller's email (e.g., Alice's email) required changes in multiple rows, increasing the risk of inconsistency.
- *After normalization:* Seller information is maintained in a single record within the **SELLER** table, so updates are made only once.

Deletion Anomaly

- *Before normalization:* Deleting the last transaction of a product could result in the loss of that product's information.
- *After normalization:* Product data is stored separately in the **PRODUCT** table, ensuring it remains intact even if related transactions are removed.